DevOps Services Setup Documentation

📋 **Date:** February 9, 2026
Server IP: 10.7.157.105
Services deployed: OpenProject, Prometheus, Grafana, Alertmanager, Node Exporter, cAdvisor

1. OpenProject Installation

What is OpenProject?

OpenProject is a **self-hosted project management tool** (similar to Jira). It provides task tracking, Gantt charts, Agile boards, time tracking, and team collaboration features. We use it to manage and plan our project work.

Directory Structure

```
~/online/openproject/
├── docker-compose.yml    # Defines the OpenProject container
└── .env                  # Contains secrets (passwords, key
s)
```

Step 1: Create the directory

```
mkdir -p ~/online/openproject
```

This creates the `openproject` folder inside `~/online/`. The `-p` flag means "create parent directories if they don't exist" — so it also created `~/online/` since it didn't exist.

Step 2: Create the .env file

```
cd ~/online/openproject
```

File: `~/online/openproject/.env`

```
OPENPROJECT_SECRET_KEY_BASE=e08e461e674d272a8318546948abc1363
035ebfd1ee7ee3fb7135a96b0414ae0b82486f3b468597a617e9aa260e47a
13653e6413c8786f77cc27d251b2dd5f72
ADMIN_PASSWORD=Admin@SecurePass2026
```

Explanation:

- `OPENPROJECT_SECRET_KEY_BASE` — A random 128-character hex string generated with `openssl rand -hex 64`. This is used by Rails (OpenProject's framework) to encrypt cookies and session data. Every deployment should have a unique one.
- `ADMIN_PASSWORD` — The password for the default `admin` user account.

Step 3: Create docker-compose.yml

File: `~/online/openproject/docker-compose.yml`

```
services:
  openproject:
    image: openproject/openproject:14
    container_name: openproject
    restart: always
    environment:
      - OPENPROJECT_SECRET_KEY_BASE=${OPENPROJECT_SECRET_KEY_
BASE}
      - OPENPROJECT_HOST__NAME=10.7.157.105
      - OPENPROJECT_HTTPS=false
      - OPENPROJECT_DEFAULT__LANGUAGE=en
      - OPENPROJECT_SEED_ADMIN_USER_PASSWORD=${ADMIN_PASSWOR
D}
```

```
      - OPENPROJECT_SEED_ADMIN_USER_NAME=admin
      - OPENPROJECT_SEED_ADMIN_USER_MAIL=admin@openproject.lo
cal
    volumes:
      - openproject_pgdata:/var/openproject/pgdata
      - openproject_assets:/var/openproject/assets
    ports:
      - "8083:8080"

volumes:
  openproject_pgdata:
  openproject_assets:
```

▼ Line-by-line explanation

Line	What it does
`image: openproject/openproject:14`	Uses the official OpenProject Docker image, version 14
`container_name: openproject`	Names the container "openproject" (for docker logs, etc.)
`restart: always`	Auto-restarts if it crashes or the server reboots
`OPENPROJECT_SECRET_KEY_BASE=$\{...\}`	Reads the secret key from the .env file
`OPENPROJECT_HOST__NAME=10.7.157.105`	Tells OpenProject what hostname/IP it's reachable at. Double underscore maps to a dot in Rails config
`OPENPROJECT_HTTPS=false`	No HTTPS directly on OpenProject (Traefik handles SSL)
`openproject_pgdata:/var/openproject/pgdata`	Docker volume for PostgreSQL data. Persists across restarts
`openproject_assets:/var/openproject/assets`	Docker volume for uploaded files and assets
`"8083:8080"`	Maps host port 8083 to container port 8080. We use 8083 because 8080

Line	What it does
	(Adminer), 8081 (GitLab internal), 8082 (GitLab HTTP) are taken

Step 4: Start OpenProject

```
cd ~/online/openproject
docker compose up -d
```

The `-d` flag runs it in "detached" mode (background). On first start, OpenProject runs database migrations which takes 1-2 minutes before the web UI becomes available.

Access

Field	Value
URL	`http://10.7.157.105:8083`
Username	`admin`
Password	`Admin@SecurePass2026`
new pswf	Koussay.1409

---

2. Monitoring Stack Installation

What is the Monitoring Stack?

A set of **5 services** working together to collect, store, visualize, and alert on system metrics:

Service	Role
Prometheus	The "brain" — scrapes (collects) metrics from all targets every 15 seconds and stores them in a time-series database
Grafana	The "dashboard" — connects to Prometheus and displays beautiful graphs and charts

Service	Role
Alertmanager	The "alarm" — receives alerts from Prometheus and routes them (email, Slack, etc.)
Node Exporter	The "host spy" — exposes the VPS machine's metrics (CPU, RAM, disk, network) so Prometheus can scrape them
cAdvisor	The "container spy" — exposes Docker container metrics (per-container CPU, memory, I/O) so Prometheus can scrape them

How they work together:

```
Node Exporter ──metrics──> Prometheus ──alerts──> Alertmanage
r
cAdvisor ─────metrics──> Prometheus ──data────> Grafana (dash
boards)
Jenkins ──────metrics──> Prometheus
GitLab ───────metrics──> Prometheus
```

Directory Structure

```
~/online/monitoring/
├── docker-compose.yml              # Defines all 5 container
s
├── .env                            # Grafana admin password
├── prometheus/
│   ├── prometheus.yml              # What to scrape and how
often
│   ├── web.yml                     # Optional basic auth for
Prometheus UI
│   └── alerts/                     # Alert rules (empty, rea
dy for rules)
├── grafana/
│   └── provisioning/               # Auto-provisioning dashb
oards/datasources
└── alertmanager/
    ├── alertmanager.yml            # Alert routing configura
```

```
tion
    └── web.yml                     # Optional basic auth for
Alertmanager UI
```

Step 1: Create the directory structure

```
mkdir -p ~/online/monitoring/{prometheus/alerts,grafana/provi
sioning,alertmanager}
```

This single command creates the entire tree. The `{a,b,c}` syntax is bash brace expansion — it creates `prometheus/alerts`, `grafana/provisioning`, and `alertmanager` all at once.

Step 2: Create the .env file

File: `~/online/monitoring/.env`

```
GRAFANA_ADMIN_PASSWORD=Grafana@SecurePass2026
```

Step 3: Create docker-compose.yml

File: `~/online/monitoring/docker-compose.yml`

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: always
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--web.console.libraries=/etc/prometheus/console_libr
aries'
      - '--web.console.templates=/etc/prometheus/consoles'
      - '--web.enable-lifecycle'
```

```
      - '--web.external-url=http://10.7.157.105:9090'
      - '--web.config.file=/etc/prometheus/web.yml'
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheu
s.yml:ro
      - ./prometheus/web.yml:/etc/prometheus/web.yml:ro
      - ./prometheus/alerts:/etc/prometheus/alerts:ro
      - prometheus_data:/prometheus
    networks:
      - monitoring-internal
    ports:
      - "9090:9090"

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: always
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
      - GF_SERVER_ROOT_URL=http://10.7.157.105:3001
      - GF_USERS_ALLOW_SIGN_UP=false
      - GF_SECURITY_DISABLE_GRAVATAR=true
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    networks:
      - monitoring-internal
    ports:
      - "3001:3000"
    depends_on:
      - prometheus

  alertmanager:
    image: prom/alertmanager:latest
    container_name: alertmanager
```

```
    restart: always
    command:
      - '--config.file=/etc/alertmanager/alertmanager.yml'
      - '--storage.path=/alertmanager'
      - '--web.external-url=http://10.7.157.105:9093'
      - '--web.config.file=/etc/alertmanager/web.yml'
    volumes:
      - ./alertmanager/alertmanager.yml:/etc/alertmanager/ale
rtmanager.yml:ro
      - ./alertmanager/web.yml:/etc/alertmanager/web.yml:ro
      - alertmanager_data:/alertmanager
    networks:
      - monitoring-internal
    ports:
      - "9093:9093"

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    restart: always
    command:
      - '--path.procfs=/host/proc'
      - '--path.rootfs=/rootfs'
      - '--path.sysfs=/host/sys'
      - '--collector.filesystem.mount-points-exclude=^/(sys|p
roc|dev|host|etc)($$|/)'
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    networks:
      - monitoring-internal
    expose:
      - "9100"

  cadvisor:
```

```
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
    restart: always
    privileged: true
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
      - /dev/disk/:/dev/disk:ro
    networks:
      - monitoring-internal
    expose:
      - "8080"

volumes:
  prometheus_data:
  grafana_data:
  alertmanager_data:

networks:
  monitoring-internal:
    driver: bridge
```

▼ Key concepts explained

Concept	Explanation
`command: \[...\]`	Override the default startup command. For Prometheus, we pass flags like where to find the config, where to store data, etc.
`volumes: ./ file:/container/path:ro`	Mounts a local file into the container. `:ro` means "read-only" — the container can read the config but can't modify it
`volumes: name:/path` (no ./)	A Docker managed volume — data persists even if the container is deleted

Concept	Explanation
`networks: monitoring-internal`	All 5 containers share a private Docker network. They can reach each other by container name (e.g., prometheus can reach alertmanager:9093)
`ports: "9090:9090"`	Exposes the port to the outside world (host:container)
`expose: "9100"`	Only accessible within the Docker network, NOT from outside. Node Exporter and cAdvisor don't need to be publicly accessible
`privileged: true` (cAdvisor)	cAdvisor needs privileged access to read Docker and system info
`depends_on: prometheus` (Grafana)	Grafana won't start until Prometheus is running first
`--web.enable-lifecycle`	Allows reloading Prometheus config via HTTP POST without restarting the container
`--web.external-url`	Tells the service what its public URL is (used for generating links in alerts)
`$$` in exclude pattern	In docker-compose, √x New equation escapes to a literal $ (regex end-of-string anchor)

Step 4: Create prometheus.yml (Scrape Configuration)

File: `~/online/monitoring/prometheus/prometheus.yml`

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'my-monitor'

alerting:
  alertmanagers:
    - static_configs:
        - targets:
          - alertmanager:9093
```

```yaml
rule_files:
  - /etc/prometheus/alerts/*.yml

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']

  - job_name: 'jenkins'
    metrics_path: /prometheus
    static_configs:
      - targets: ['10.7.157.105:8089']

  - job_name: 'gitlab'
    metrics_path: /-/metrics
    static_configs:
      - targets: ['10.7.157.105:8082']
```

▼ Scrape config explanation

Section	What it does
`scrape_interval: 15s`	Prometheus collects metrics from every target every 15 seconds
`evaluation_interval: 15s`	Prometheus evaluates alert rules every 15 seconds
`external_labels`	Labels added to all metrics (useful when federating multiple Prometheus instances)

Section	What it does
`alerting > alertmanagers`	Tells Prometheus where Alertmanager lives. Uses Docker network name alertmanager on port 9093
`rule_files`	Path to alert rule files. Any .yml file in the alerts/ folder will be loaded
`job_name: 'prometheus'`	Prometheus scrapes itself (meta-monitoring)
`job_name: 'node-exporter'`	Scrapes Node Exporter inside the Docker network (not exposed publicly)
`job_name: 'cadvisor'`	Scrapes cAdvisor inside the Docker network
`job_name: 'jenkins'`	Scrapes Jenkins at /prometheus endpoint. Uses VPS IP because Jenkins runs outside this Docker network
`job_name: 'gitlab'`	Scrapes GitLab at /-/metrics endpoint. Uses VPS IP since GitLab is a separate Docker container

Step 5: Create alertmanager.yml (Alert Routing)

File: `~/online/monitoring/alertmanager/alertmanager.yml`

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'severity']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'default-receiver'
  routes:
    - match:
        severity: critical
      receiver: 'critical-receiver'
      repeat_interval: 1h

receivers:
  - name: 'default-receiver'
```

```
  - name: 'critical-receiver'

inhibit_rules:
  - source_match:
      severity: 'critical'
    target_match:
      severity: 'warning'
    equal: ['alertname', 'instance']
```

▼ Alertmanager config explanation

Setting	What it does
`resolve_timeout: 5m`	If an alert stops firing, wait 5 minutes before marking it as "resolved"
`group_by`	Group alerts with the same name and severity into one notification (avoids spam)
`group_wait: 30s`	Wait 30s after first alert in a group before sending (collects related alerts)
`group_interval: 5m`	Wait 5 minutes before sending updates for the same group
`repeat_interval: 4h`	Re-send the same alert every 4 hours if still firing
`severity: critical` `route`	Critical alerts go to critical-receiver and repeat every 1 hour (more urgent)
`inhibit_rules`	If a critical alert fires, suppress the corresponding warning alert (no duplicate noise)
Receivers are empty	No email/Slack configured yet — they just log. Add webhook/email config later

Step 6: Create web.yml files (Optional Auth)

File: `~/online/monitoring/prometheus/web.yml` and `~/online/monitoring/alertmanager/web.yml`

```
# Basic auth configuration (optional)
# Uncomment and configure if you want password protection:
```

```
# basic_auth_users:
#   admin: $2y$10$...  # bcrypt hash of password
```

These are placeholder files. Prometheus and Alertmanager require the web.yml file to exist (we reference it in `--web.config.file`), but it can be mostly empty. You can add basic authentication later.

Step 7: Start the Monitoring Stack

```
cd ~/online/monitoring
docker compose up -d
```

This pulls 5 Docker images and starts 5 containers.

Access

Service	URL	Username	Password
Prometheus	`http://10.7.157.105:9090`	—	—
Grafana	`http://10.7.157.105:3001`	`admin`	`Grafana@SecurePass2026`
Alertmanager	`http://10.7.157.105:9093`	—	—
Node Exporter	Internal only	—	—
cAdvisor	Internal only	—	—

Prometheus Scrape Targets Status

Target	Status	Notes
prometheus (self)	UP	Self-monitoring
node-exporter	UP	Host machine metrics working
cadvisor	UP	Docker container metrics working
gitlab	DOWN	GitLab has prometheus_monitoring['enable'] = false — needs to be enabled
jenkins	DOWN	Jenkins is not running yet — will work once started with Prometheus plugin

3. Verification Commands

Check all running containers:

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Port
s}}"
```

Check Prometheus targets:

```
curl -s http://localhost:9090/api/v1/targets | python3 -m jso
n.tool
```

Check service health:

```
curl -s http://localhost:9090/-/healthy     # Prometheus
curl -s http://localhost:3001/api/health    # Grafana
curl -s http://localhost:9093/-/healthy     # Alertmanager
curl -s http://10.7.157.105:8083           # OpenProject
```

View logs:

```
docker logs -f openproject      # OpenProject logs
docker logs -f prometheus       # Prometheus logs
docker logs -f grafana          # Grafana logs
docker logs -f alertmanager     # Alertmanager logs
```

Stop/restart services:

```
# OpenProject
cd ~/online/openproject && docker compose down      # Stop
cd ~/online/openproject && docker compose up -d      # Start

# Monitoring Stack (all 5 containers)
```

```
cd ~/online/monitoring && docker compose down
cd ~/online/monitoring && docker compose up -d
```

---

4. Full Port Map (All Services on VPS)

Port	Service	Status
22	SSH	Running
2222	GitLab SSH	Running
3001	Grafana	Running
5432	PostgreSQL (app database)	Running
6443	k3s Kubernetes API	Running
8080	Adminer (DB management)	Running
8082	GitLab HTTP	Running
8083	OpenProject	Running
8443	GitLab HTTPS	Running
9090	Prometheus	Running
9093	Alertmanager	Running
30080	AWX (Ansible)	Running

---

5. Next Steps: Grafana First-Time Setup

After logging into Grafana at `http://10.7.157.105:3001`:

1. Go to **Connections** > **Data Sources** > **Add data source**
2. Select **Prometheus**
3. Set URL to `http://prometheus:9090` (uses the Docker internal network name)
4. Click **Save & Test** — should show "Data source is working"
5. Go to **Dashboards** > **Import** and use these popular dashboard IDs:
    - `1860` — Node Exporter Full (host metrics)

- `193` — Docker monitoring (cAdvisor)

---

6. Kubernetes Services (docker-services.yaml)

What are these?

Traefik (the reverse proxy built into k3s) lives inside Kubernetes. Our Docker containers (Jenkins, GitLab, OpenProject, etc.) live **outside** Kubernetes, on the host machine. Kubernetes Services bridge this gap — they tell Traefik "when you get traffic for this service, forward it to the host IP on this port."

Two objects are needed per service:

1. **Service** (without a selector) — defines the port to listen on inside the cluster
2. **Endpoints** — tells the Service where to actually send the traffic (the host IP `10.7.157.105`)

Without Endpoints, a Service with no selector is like a phone number with no one to answer.

File Location

`~/projet-fin-etude-main/docker-services.yaml`

Full Configuration

```
# Kubernetes Services + Endpoints for Docker containers
# These allow Traefik (k3s ingress) to route traffic to Docke
r services running on the host
# The Endpoints point to the node IP (10.7.157.105) where Doc
ker containers expose their ports

---
# Jenkins Service (port 8089)
apiVersion: v1
```

```yaml
kind: Service
metadata:
  name: jenkins-docker
  namespace: default
spec:
  ports:
    - port: 8089
      targetPort: 8089
---
apiVersion: v1
kind: Endpoints
metadata:
  name: jenkins-docker
  namespace: default
subsets:
  - addresses:
      - ip: 10.7.157.105
    ports:
      - port: 8089

---
# GitLab Service (port 8082)
apiVersion: v1
kind: Service
metadata:
  name: gitlab-docker
  namespace: default
spec:
  ports:
    - port: 8082
      targetPort: 8082
---
apiVersion: v1
kind: Endpoints
metadata:
  name: gitlab-docker
```

```
  namespace: default
subsets:
  - addresses:
      - ip: 10.7.157.105
    ports:
      - port: 8082

---
# OpenProject Service (port 8083)
apiVersion: v1
kind: Service
metadata:
  name: openproject-docker
  namespace: default
spec:
  ports:
    - port: 8083
      targetPort: 8083
---
apiVersion: v1
kind: Endpoints
metadata:
  name: openproject-docker
  namespace: default
subsets:
  - addresses:
      - ip: 10.7.157.105
    ports:
      - port: 8083

---
# Prometheus Service (port 9090)
apiVersion: v1
kind: Service
metadata:
  name: prometheus-docker
```

```
  namespace: default
spec:
  ports:
    - port: 9090
      targetPort: 9090
---
apiVersion: v1
kind: Endpoints
metadata:
  name: prometheus-docker
  namespace: default
subsets:
  - addresses:
      - ip: 10.7.157.105
    ports:
      - port: 9090

---
# Grafana Service (port 3001)
apiVersion: v1
kind: Service
metadata:
  name: grafana-docker
  namespace: default
spec:
  ports:
    - port: 3001
      targetPort: 3001
---
apiVersion: v1
kind: Endpoints
metadata:
  name: grafana-docker
  namespace: default
subsets:
  - addresses:
```

```
      - ip: 10.7.157.105
    ports:
      - port: 3001

---
# Alertmanager Service (port 9093)
apiVersion: v1
kind: Service
metadata:
  name: alertmanager-docker
  namespace: default
spec:
  ports:
    - port: 9093
      targetPort: 9093
---
apiVersion: v1
kind: Endpoints
metadata:
  name: alertmanager-docker
  namespace: default
subsets:
  - addresses:
      - ip: 10.7.157.105
    ports:
      - port: 9093
```

Apply Command

```
kubectl apply -f ~/projet-fin-etude-main/docker-services.yaml
```

Verify

```
# Check services
kubectl get svc -n default | grep docker

# Check endpoints
kubectl get endpoints -n default | grep docker
```

Result

Service	Cluster IP	Forwards to
jenkins-docker	10.43.33.124:8089	10.7.157.105:8089
gitlab-docker	10.43.147.129:8082	10.7.157.105:8082
openproject-docker	10.43.13.30:8083	10.7.157.105:8083
prometheus-docker	10.43.12.79:9090	10.7.157.105:9090
grafana-docker	10.43.222.65:3001	10.7.157.105:3001
alertmanager-docker	10.43.182.2:9093	10.7.157.105:9093